

Seguimiento Programación Orientadas A Objetos

Nombre De los Estudiantes:

**Salvador Eduardo Flores
Diego Fernando Culma Gaviria**

Nombre Del Profesor:

Walter Hugo Arboleda Mazo

**Universidad Nacional De Colombia Sede Medellín
2026-1**

Ejercicio 6.0

Enlace:

Código:

```
import os
import tkinter as tk
from tkinter import messagebox
from tkinter import ttk

class GestorArchivo:

    def __init__(self, nombre_archivo):
        self._nombre_archivo = nombre_archivo
        self._verificar_archivo()

    def _verificar_archivo(self):
        if not os.path.exists(self._nombre_archivo):
            with open(self._nombre_archivo, "w", encoding="utf-8") as f:
                pass

    def _leer_todos(self):
        self._verificar_archivo()
        contactos = {}
        with open(self._nombre_archivo, "r", encoding="utf-8") as f:
            for linea in f:
                linea = linea.strip()
                if linea and "!" in linea:
                    nombre, numero = linea.split("!", 1)
                    contactos[nombre.strip()] = numero.strip()
        return contactos

    def _guardar_todos(self, contactos):
        with open(self._nombre_archivo, "w", encoding="utf-8") as f:
            for nombre, numero in contactos.items():
                f.write(f"{nombre}!{numero}\n")

#Create

    def crear(self, nombre, numero):
        if not nombre or not numero:
            return False, "Por favor llene ambos campos (Nombre y Número)."
```

```

    if "!" in nombre or "!" in numero:
        return False, "El carácter '!' no está permitido en los campos."

    contactos = self._leer_todos()

    if nombre in contactos:
        return False, f"El contacto '{nombre}' ya existe. Use Update para modificarlo."

    contactos[nombre] = numero
    self._guardar_todos(contactos)
    return True, f"Contacto '{nombre}' creado correctamente."

# Read

def leer(self):
    contactos = self._leer_todos()

    if not contactos:
        return {}
    return contactos

# Update

def actualizar(self, nombre, nuevo_numero):
    if not nombre or not nuevo_numero:
        return False, "Asegúrese de que Nombre y Número tengan datos."

    if "!" in nombre or "!" in nuevo_numero:
        return False, "El carácter '!' no está permitido en los campos."

    contactos = self._leer_todos()

    if nombre not in contactos:
        return False, f"El contacto '{nombre}' no existe. No se puede actualizar."

    contactos[nombre] = nuevo_numero

    self._guardar_todos(contactos)
    return True, f"Contacto '{nombre}' actualizado correctamente."

# Delete

def eliminar(self, nombre):
    if not nombre:
        return False, "Escriba o seleccione el Nombre del contacto a eliminar."

```

```

    contactos = self._leer_todos()

    if nombre not in contactos:
        return False, f"El contacto '{nombre}' no se encuentra en el archivo."

    del contactos[nombre]

    self._guardar_todos(contactos)
    return True, f"Contacto '{nombre}' eliminado correctamente."

class VentanaPrincipal(tk.Tk):

    def __init__(self):
        super().__init__()

        self.title("Contactos CRUD")
        self.resizable(False, False)

        # Centrar ventana en pantalla
        self.update_idletasks()
        ancho, alto = 440, 500
        x = (self.winfo_screenwidth() // 2) - (ancho // 2)
        y = (self.winfo_screenheight() // 2) - (alto // 2)
        self.geometry(f"{ancho}x{alto}+{x}+{y}")

        style = ttk.Style()
        style.theme_use("clam")

        self._gestor = GestorArchivo("contacts.txt")

        self._crear_widgets()
        self._cargar_lista()

    #crear widgets

    def _crear_widgets(self):

        # --- SECCIÓN: CAMPOS DE ENTRADA ---
        frame_campos = ttk.LabelFrame(self, text=" Información del Contacto ",
padding=10)
        frame_campos.pack(fill="x", padx=15, pady=10)
        frame_campos.columnconfigure(1, weight=1)

        ttk.Label(frame_campos, text="Nombre:").grid(row=0, column=0, sticky="w",
pady=5, padx=5)
        self.entry_nombre = ttk.Entry(frame_campos, font=("Arial", 10))
        self.entry_nombre.grid(row=0, column=1, sticky="ew", pady=5, padx=5)

```

```

        ttk.Label(frame_campos, text="Número:").grid(row=1, column=0, sticky="w",
pady=5, padx=5)
        self.entry_numero = ttk.Entry(frame_campos, font=("Arial", 10))
        self.entry_numero.grid(row=1, column=1, sticky="ew", pady=5, padx=5)

# --- SECCIÓN: BOTONES CRUD ---
        frame_botones = ttk.Frame(self, padding=5)
        frame_botones.pack(fill="x", padx=15)
        frame_botones.columnconfigure((0, 1, 2, 3), weight=1)

# Create
        btn_create = ttk.Button(frame_botones, text="Create",
command=self._accion_create)
        btn_create.grid(row=0, column=0, padx=4, pady=5, sticky="ew")

# Read
        btn_read = ttk.Button(frame_botones, text="Read",
command=self._accion_read)
        btn_read.grid(row=0, column=1, padx=4, pady=5, sticky="ew")

# Update
        btn_update = ttk.Button(frame_botones, text="Update",
command=self._accion_update)
        btn_update.grid(row=0, column=2, padx=4, pady=5, sticky="ew")

# Delete
        btn_delete = ttk.Button(frame_botones, text="Delete",
command=self._accion_delete)
        btn_delete.grid(row=0, column=3, padx=4, pady=5, sticky="ew")

# --- BOTONES DE SOPORTE ---
        frame_soporte = ttk.Frame(self, padding=2)
        frame_soporte.pack(fill="x", padx=15)
        frame_soporte.columnconfigure((0, 1), weight=1)

        btn_seleccionar = ttk.Button(frame_soporte, text="Seleccionar contacto",
command=self._seleccionar)
        btn_seleccionar.grid(row=0, column=0, padx=4, pady=3, sticky="ew")

        btn_limpiar = ttk.Button(frame_soporte, text="Limpiar campos",
command=self._limpiar_campos)
        btn_limpiar.grid(row=0, column=1, padx=4, pady=3, sticky="ew")

# --- SECCIÓN: LISTA DE CONTACTOS (READ visual) ---
        frame_lista = ttk.LabelFrame(self, text=" Lista de Contactos ", padding=10)
        frame_lista.pack(fill="both", expand=True, padx=15, pady=10)

```

```

scrollbar = ttk.Scrollbar(frame_lista)
scrollbar.pack(side="right", fill="y")

self.lista = tk.Listbox(
    frame_lista,
    font=("Arial", 10),
    yscrollcommand=scrollbar.set,
    selectmode=tk.SINGLE
)
self.lista.pack(side="left", fill="both", expand=True)
scrollbar.config(command=self.lista.yview)

def _cargar_lista(self):
    self.lista.delete(0, tk.END)
    contactos = self._gestor.leer()
    for nombre in sorted(contactos.keys()):
        numero = contactos[nombre]
        self.lista.insert(tk.END, f"{nombre} → {numero}")

def _limpiar_campos(self):
    self.entry_nombre.delete(0, tk.END)
    self.entry_numero.delete(0, tk.END)
    self.lista.selection_clear(0, tk.END)

def _seleccionar(self):
    try:
        seleccion = self.lista.curselection()
        if not seleccion:
            messagebox.showwarning("Advertencia", "Seleccione un contacto de la
lista.")
        return

        linea = self.lista.get(seleccion[0])
        nombre, numero = linea.split(" → ", 1)

        self._limpiar_campos()
        self.entry_nombre.insert(0, nombre.strip())
        self.entry_numero.insert(0, numero.strip())

    except Exception as e:
        messagebox.showerror("Error", f"Ocurrió un error: {e}")

def _accion_create(self):
    nombre = self.entry_nombre.get().strip()
    numero = self.entry_numero.get().strip()

```

```

    exito, mensaje = self._gestor.crear(nombre, numero)

    if exito:
        messagebox.showinfo("Éxito", mensaje)
        self._cargar_lista()
        self._limpiar_campos()
    else:
        messagebox.showwarning("Advertencia", mensaje)

def _accion_read(self):
    contactos = self._gestor.leer()
    self._cargar_lista()

    if not contactos:
        messagebox.showinfo("Read", "No hay contactos en el archivo.")
    else:
        resumen = ""
        for nombre, numero in sorted(contactos.items()):
            resumen += f"Nombre: {nombre}\nNúmero: {numero}\n\n"
        messagebox.showinfo("Contactos en archivo", resumen.strip())

def _accion_update(self):
    nombre = self.entry_nombre.get().strip()
    numero = self.entry_numero.get().strip()

    exito, mensaje = self._gestor.actualizar(nombre, numero)

    if exito:
        messagebox.showinfo("Éxito", mensaje)
        self._cargar_lista()
        self._limpiar_campos()
    else:
        messagebox.showerror("Error", mensaje)

def _accion_delete(self):
    nombre = self.entry_nombre.get().strip()

    if not nombre:
        messagebox.showwarning("Campo vacío", "Escriba o seleccione el Nombre del contacto a eliminar.")
        return

    confirmar = messagebox.askyesno(
        "Confirmar eliminación",
        f"¿Está seguro de que desea eliminar a '{nombre}'?"
    )

```

```

    if confirmar:
        exito, mensaje = self._gestor.eliminar(nombre)
        if exito:
            messagebox.showinfo("Éxito", mensaje)
            self._cargar_lista()
            self._limpiar_campos()
        else:
            messagebox.showerror("Error", mensaje)

def main():
    app = VentanaPrincipal()
    app.mainloop()

main()

```

Diagrama de Clases:

- nombre archivo

+ verificar archivo ()
+ leer todos ()
+ guardar todos ()
+ crear ()
+ leer ()
+ actualizar ()
+ eliminar ()

Clase: VentanaPrincipal (hereda de tk.Tk)

- gestor
- entry nombre
- entry numero
- lista

+ crear widgets ()
+ cargar lista ()
+ limpiar campos ()
+ seleccionar ()
+ accion create ()
+ accion read ()
+ accion update ()
+ accion delete ()

Caso1 Crear:

#Create

```
def crear(self, nombre, numero):
    if not nombre or not numero:
        return False, "Por favor llene ambos campos (Nombre y Número)."

    if "!" in nombre or "!" in numero:
        return False, "El carácter '!' no está permitido en los campos."

    contactos = self._leer_todos()

    if nombre in contactos:
        return False, f"El contacto '{nombre}' ya existe. Use Update para modificarlo."

    contactos[nombre] = numero
    self._guardar_todos(contactos)
    return True, f"Contacto '{nombre}' creado correctamente."
```

Diagrama de Clases:

- validar campos no vacios
- validar carácter "!" no permitido
- leer contactos existentes
- validar que nombre no exista
- agregar contacto
- guardar contactos en archivo

+ retorna (True, "creado")
+ retorna (False, "error")

Información del Contacto

Nombre:

Número:

Create Read Update Delete

Seleccionar contacto Limpiar campos

Lista de Contactos

- Diego → 15
- Mauricio → 82
- Messi → 10
- Rayo McQueen → 95

Caso2 Leer:

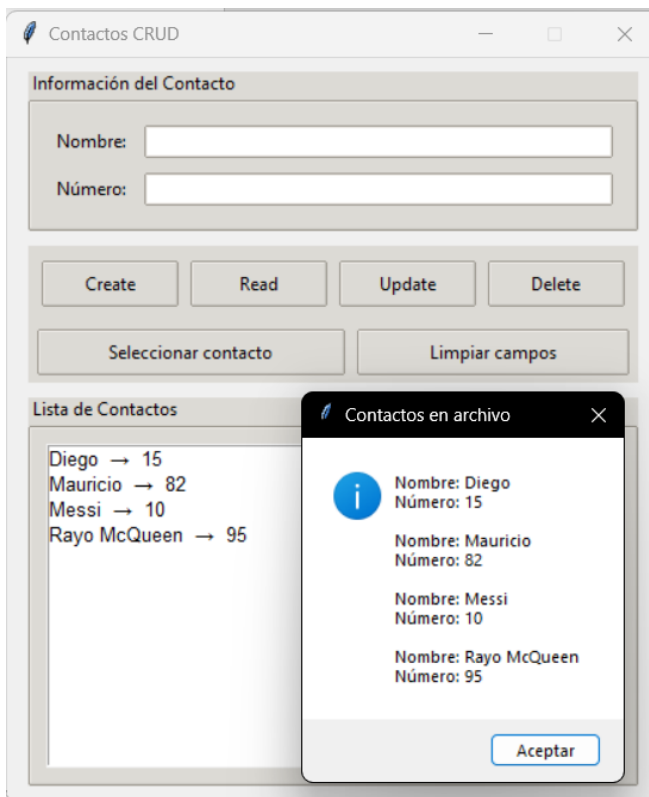
Read

```
def leer(self):  
    contactos = self._leer_todos()  
  
    if not contactos:  
        return {}  
    return contactos
```

Diagrama de Clases:

-
- leer contactos existentes
 - validar si esta vacio
-

- + retorna {} (si esta vacio)
- + retorna contactos (diccionario)



Caso3 Actualizar:

Update

```
def actualizar(self, nombre, nuevo_numero):
    if not nombre or not nuevo_numero:
        return False, "Asegúrese de que Nombre y Número tengan datos."

    if "!" in nombre or "!" in nuevo_numero:
        return False, "El carácter '!' no está permitido en los campos."

    contactos = self._leer_todos()

    if nombre not in contactos:
        return False, f"El contacto '{nombre}' no existe. No se puede actualizar."

    contactos[nombre] = nuevo_numero

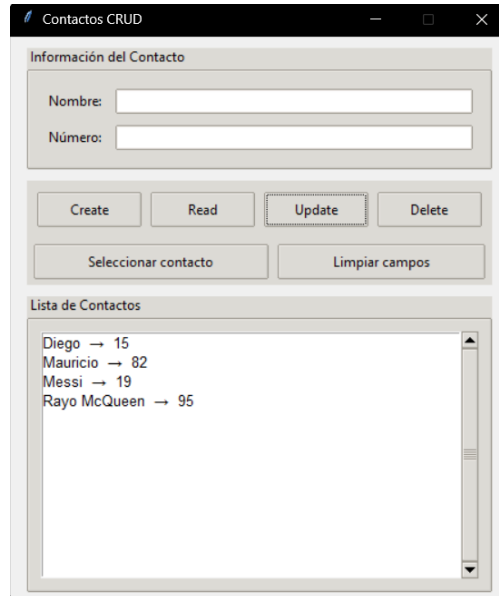
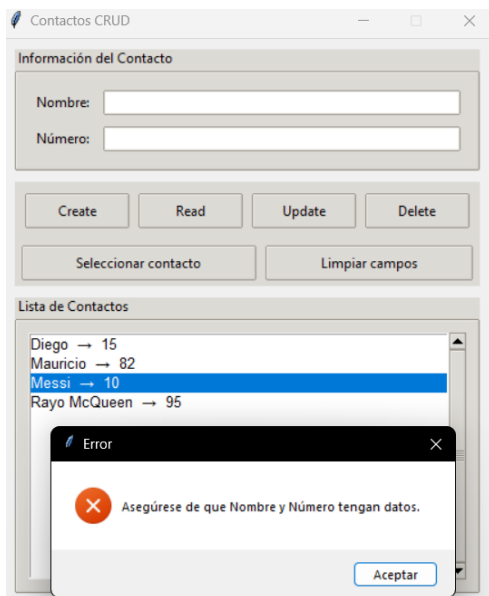
    self._guardar_todos(contactos)
    return True, f"Contacto '{nombre}' actualizado correctamente."
```

Diagrama de Clases:

- validar campos no vacios
- validar caracter "!" no permitido
- leer contactos existentes
- validar que nombre SI exista
- actualizar numero del contacto
- guardar contactos en archivo

+ retorna (True, "actualizado")

+ retorna (False, "error")



Caso4 Eliminar:

Delete

```
def eliminar (self, nombre):  
    if not nombre:  
        return False, "Escriba o seleccione el Nombre del contacto a eliminar."
```

```
    contactos = self._leer_todos()
```

```
    if nombre not in contactos:  
        return False, f"El contacto '{nombre}' no se encuentra en el archivo."
```

```
    del contactos[nombre]
```

```
    self._guardar_todos(contactos)
```

```
    return True, f"Contacto '{nombre}' eliminado correctamente."
```

Diagrama de Clases:

- validar campo nombre no vacio
- leer contactos existentes
- validar que nombre exista
- eliminar contacto del diccionario
- guardar contactos en archivo

+ retorna (True, "eliminado")

+ retorna (False, "error")

